

Reforça+: Reforço Escolar Comunitário (PWA)

Prática Extensionista IV, Avaliação 1
Modelagem e Arquitetura do Sistema de Informação

1. Integrantes do grupo

- **Vinicios Andrei Mensen**, RA 445509

2. Link do repositório Git (Prática Extensionista IV)

<https://github.com/Vorceek/reforcamaais>

O repositório contém o README com a documentação do projeto, as fontes dos diagramas em PlantUML (docs/diagramas), as imagens renderizadas (docs/imagens), a justificativa da infraestrutura (docs/infraestrutura.md) e o pipeline de CI (.github/workflows/ci.yml).

Contexto do sistema

Alunos da rede pública em situação de vulnerabilidade têm dificuldade de acesso a reforço escolar, enquanto voluntários (universitários, professores aposentados) não dispõem de um canal organizado para oferecer ajuda. O **Reforça+** é um Progressive Web App (PWA) que conecta tutores voluntários a alunos e responsáveis, permitindo cadastro, agendamento de aulas de reforço, disponibilização de materiais com acesso offline, acompanhamento de frequência e progresso por disciplina e notificações push. Por ser PWA, funciona em qualquer smartphone sem loja de aplicativos, é instalável e resiliente a conexões instáveis, cenário comum na comunidade atendida.

Stack: React 18 + Vite + Workbox (frontend PWA); Node.js 20 + Express + Prisma (API REST, JWT); PostgreSQL 16; GitHub Actions (CI/CD); Vercel, Render e Neon (hospedagem).

3. Diagrama UML de Pacotes (Arquitetura da Aplicação)

O sistema é organizado em dois subsistemas principais. O **Frontend PWA** divide-se em ui (pages, components), state, services (api-client, auth) e pwa (service-worker, cache-offline em IndexedDB, push-notifications). O **Backend API** segue Clean Architecture em camadas: presentation (controllers, middlewares), application (usecases, dto), domain (entities, interfaces de repositories) e infrastructure (persistence com Prisma, auth JWT, notifications). As dependências (setas tracejadas «use») apontam sempre para o domínio, que não depende de nada externo; a persistência realiza («realize») as interfaces de repositório.

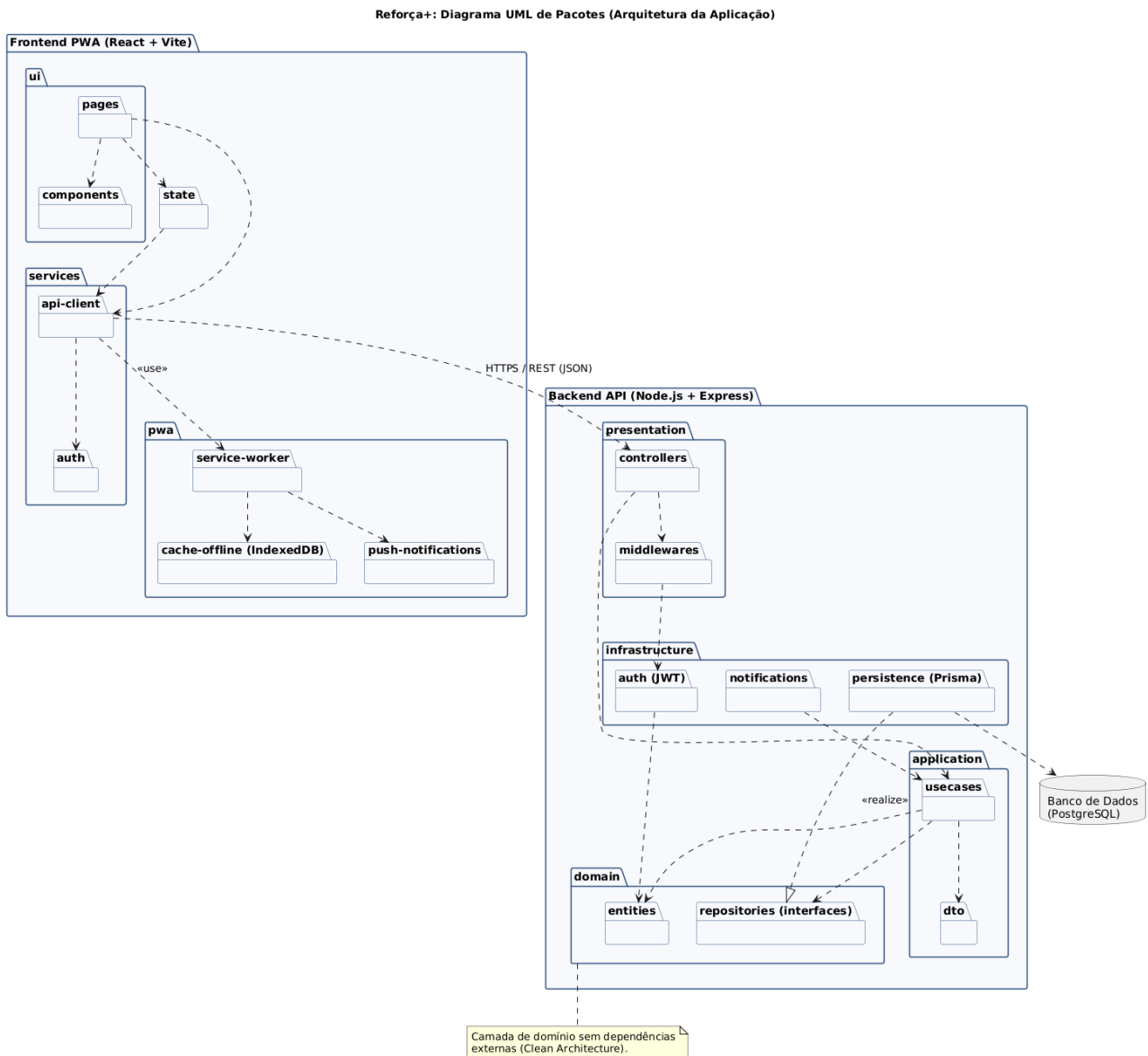


Figura 1: Diagrama UML de pacotes do Reforça+ (fonte: docs/diagramas/01-diagrama-pacotes.puml)

4. Diagrama de Arquitetura de Implantação

Diagrama UML de implantação (deployment) com nós físicos/virtuais («device», «execution environment», «cloud») e artefatos implantados. O PWA e o service worker executam no navegador do dispositivo do usuário, com dados offline em IndexedDB/Cache Storage. Os assets estáticos são servidos pela CDN da Vercel; a API Express roda em container Docker no Render e acessa o PostgreSQL gerenciado no Neon via TLS. Serviços externos: Firebase Cloud Messaging (push) e Google OAuth 2.0 (login). Todos os canais de comunicação usam HTTPS, requisito obrigatório para PWA.

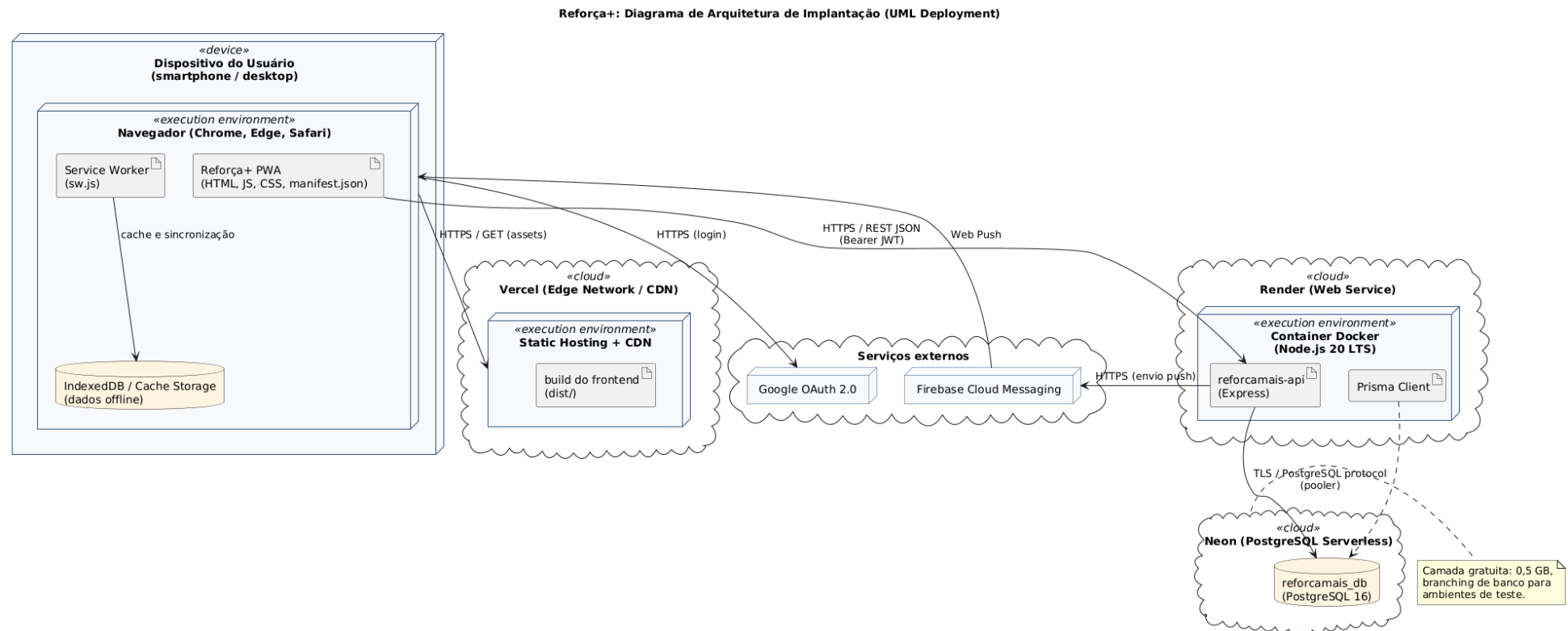


Figura 2: Diagrama de implantação do Reforça+ (fonte: docs/diagramas/02-diagrama-implantacao.puml)

5. Diagrama de Arquitetura DevOps

O ciclo DevOps é modelado em sete etapas: (1) planejamento com GitHub Issues/Projects; (2) codificação com Git e Pull Requests com revisão; (3) integração contínua no GitHub Actions (lint, testes unitários, build, auditoria PWA com Lighthouse CI e verificação de segurança (npm audit, Dependabot)); (4) entrega contínua com preview deploy por PR e migrações Prisma; (5) implantação automática na Vercel (frontend PWA), Render (API em Docker) e Neon (banco) a cada push em main; (6) operação e monitoramento com logs, Sentry e UptimeRobot; (7) feedback dos usuários da comunidade retroalimentando o backlog.

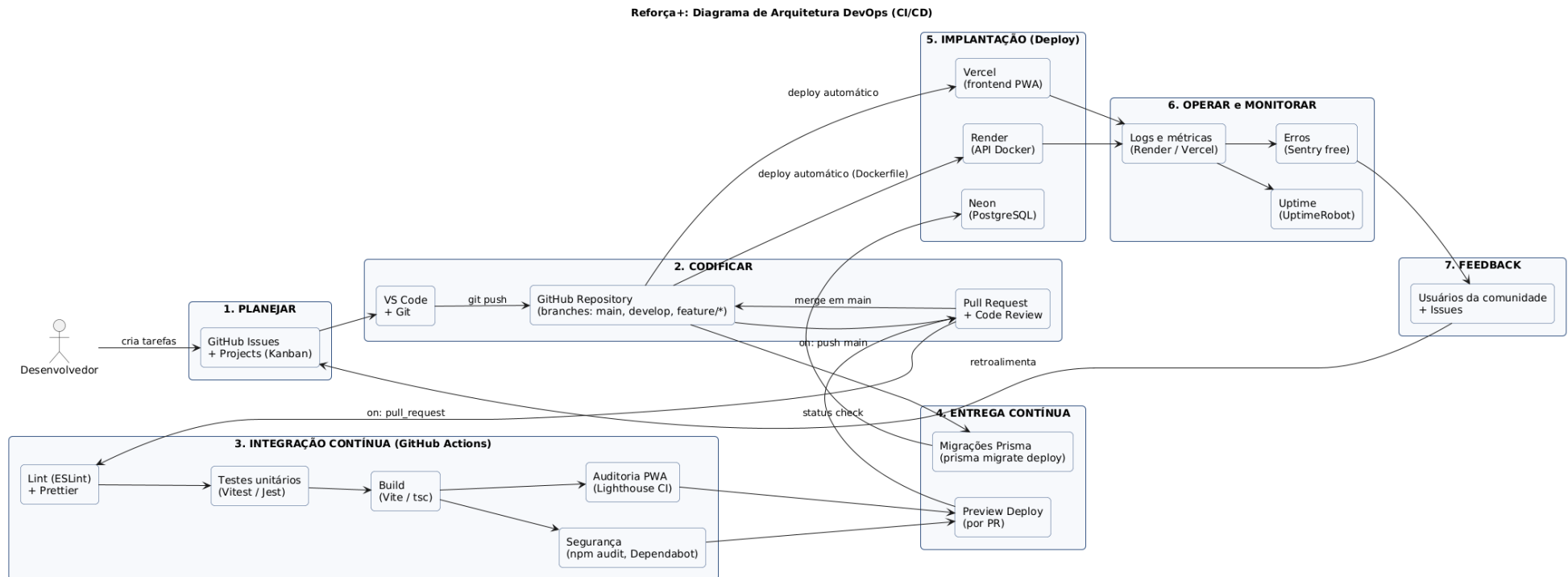


Figura 3: Diagrama de arquitetura DevOps do Reforça+ (fonte: docs/diagramas/03-diagrama-devops.puml)

6. Descrição e Justificativa da Infraestrutura de Deploy

Escolha

Arquitetura **cloud, serverless/PaaS, de custo zero**, composta por três serviços gerenciados:

Componente	Serviço / plano e função
Frontend PWA	Vercel (Hobby, gratuito): hospedagem estática com CDN global, HTTPS automático, preview por Pull Request.
API REST	Render (Web Service Free): container Docker Node.js com deploy automático a partir do GitHub.
Banco de dados	Neon (Free): PostgreSQL serverless, 0,5 GB, branching de banco para testes.
Apoio	GitHub + Actions (código e CI/CD), Firebase Cloud Messaging (push), Sentry e UptimeRobot (monitoramento), todos gratuitos.

Justificativa

Custo zero e sustentabilidade. O projeto atende uma comunidade sem orçamento. Todos os serviços têm planos gratuitos permanentes (não créditos temporários), permitindo que a solução permaneça no ar após o fim da disciplina, sem cartão de crédito.

HTTPS obrigatório para PWA. Service worker, instalação e push só funcionam sobre HTTPS. Vercel e Render emitem certificados TLS automaticamente; em self-host isso exigiria gerenciar Nginx, Certbot e renovações.

Aderência ao DevOps. Integração nativa com o GitHub: push em main gera deploy automático e cada PR ganha ambiente de preview; o Neon permite branches de banco para testes, alinhado ao pipeline modelado.

Baixa carga operacional. Não há servidor, firewall ou backup manual a manter: o provedor cuida de SO, patches e disponibilidade, liberando uma equipe pequena para focar no produto e na comunidade.

Comparação com alternativas

Alternativa	Motivo da não escolha
AWS / Azure / GCP (free tier)	Free tier expira em 12 meses ou depende de créditos estudantis; exige cartão de crédito; curva de aprendizado (IAM, VPC, EC2) desproporcional ao projeto; risco de cobranças inesperadas.
Self-host (VPS ou servidor da instituição)	Custo mensal recorrente (VPS) ou dependência do setor de TI; toda a operação (SO, TLS, backup, monitoramento) fica com a equipe; menor disponibilidade.
Firebase (Hosting + Firestore + Functions)	Banco NoSQL menos adequado ao modelo relacional (alunos, tutores, aulas, frequências); forte lock-in; Functions com chamadas externas exige plano pago.
Heroku	Plano gratuito descontinuado em 2022.
Supabase	Equivalente válido ao Neon; Neon preferido por não pausar projetos após inatividade curta e por oferecer branching. Substituível sem alterar a arquitetura.

Limitações e mitigações

O Render Free hiberna após 15 min sem tráfego (cold start $\approx$ 30 s): mitigado por ping periódico do UptimeRobot e pelo cache offline do PWA. O Neon Free limita a 0,5 GB: suficiente para dados textuais; materiais pesados ficam em links externos. A API em Docker e o PostgreSQL padrão garantem portabilidade para VPS ou outra cloud sem reescrita.

Ativação da infraestrutura

Contas criadas com login GitHub na Vercel, Render e Neon; repositório importado nas duas primeiras; projeto PostgreSQL criado no Neon com a `DATABASE_URL` configurada como variável de ambiente no Render; projeto Firebase apenas para Cloud Messaging. O checklist completo está em `docs/infraestrutura.md`.